BARBERSYSTEM

Documentação de Testes — Camada de Clientes

Coleção Postman completa com cenários de sucesso e falha

Projeto	BarberSystem — Gestão de Barbearia
Camada	Clientes (Controller + Service + Security)
Ferramenta	Postman
Base URL	http://localhost:8080
Autenticação	JWT (Bearer Token)
Data	13/10/2026

1. Visão Geral dos Endpoints

A camada de Clientes expõe 9 endpoints REST. O cadastro é público; os demais exigem autenticação JWT e autorização por papel (ROLE). A exclusão é lógica (status INATIVO).

Método	Rota	Autorização	Descrição
POST	/clientes	Público	Cadastro de cliente
GET	/clientes/lista/ativo	ADMIN / BARBEIRO	Lista clientes ativos
GET	/clientes/lista/inativo	ADMINISTRADOR	Lista clientes inativos
GET	/clientes/busca/nome	ADMIN / BARBEIRO	Busca por nome (query param)
GET	/clientes/{id}	Autenticado	Busca por ID
PATCH	/clientes/{id}	CLIENTE (próprio)	Atualiza dados cadastrais
PATCH	/clientes/{id}/senha	CLIENTE (próprio)	Altera senha
PATCH	/clientes/desativar/{id}	CLIENTE (próprio)	Desativa (exclusão lógica)
PATCH	/clientes/{id}/reativar	ADMIN	Reativa cliente

Obs.: *No controller o reativar usa* `hasRole('ADMIN')`*, enquanto o restante do sistema usa* `ADMINISTRADOR`*. Verifique a role gerada no JWT.*

2. Variáveis de Ambiente (Postman)

Crie um Environment no Postman e configure as variáveis abaixo. Após o login, armazene os tokens nas respectivas variáveis.

Variável	Exemplo / Valor	Uso
base_url	http://localhost:8080	URL base da API
token_cliente	(JWT do cliente)	Header Authorization
token_admin	(JWT do admin)	Header Authorization
token_barbeiro	(JWT do barbeiro)	Header Authorization
cliente_id	1	Path variable {id}

3. Cadastro de Cliente (POST /clientes)

Endpoint público. Valida e-mail e telefone únicos, confirmação de senha e força da senha (mín. 8 caracteres, maiúscula, minúscula, número e caractere especial).

3.1 Body de sucesso (ClienteDto)

```
{
  "nome": "João da Silva",
  "dataNascimento": "1995-05-15",
  "telefone": "(11) 98765-4321",
  "email": "joao.silva@email.com",
  "senha": "Senha@123",
  "confirmaSenha": "Senha@123"
}
```

Status esperado: ***201 Created*** *— retorna ClienteSaidaDto com id gerado.*

3.2 Cenários de falha

Cenário	Status esperado	Mensagem / Causa
E-mail já existente	400 / 409	E-mail já existente!
Telefone já existente	400 / 409	Telefone já existente!
Senha ≠ confirmaSenha	400	Senha não condiz com a confirmação
Senha fraca	400	Validação @Pattern (força mínima)
Campos obrigatórios vazios	400	Bean Validation (@NotBlank, @NotNull)
Telefone formato inválido	400	Pattern: (XX) 9XXXX-XXXX
Data de nascimento futura	400	@Past

4. Listagens

4.1 GET /clientes/lista/ativo

Roles: **ADMINISTRADOR** ou **BARBEIRO**. Header: `Authorization: Bearer {{token_admin}}`. Resposta: array de ClienteSaidaDto. Status 200.

Sem token → 401 Unauthorized. Token de CLIENTE → 403 Forbidden.

4.2 GET /clientes/lista/inativo

Role: **ADMINISTRADOR** apenas. Barbeiro ou Cliente → 403. Status 200 com lista de inativos.

5. Buscas

5.1 GET /clientes/busca/nome?nome={texto}

Roles: ADMINISTRADOR ou BARBEIRO. Busca case-insensitive (ContainingIgnoreCase). Exemplo: `{{base_url}}/clientes/busca/nome?nome=João`

5.2 GET /clientes/{id}

Qualquer usuário autenticado. ID inexistente → 404 ResourceNotFoundException ("Usuário não encontrado!").

6. Atualização de Dados (PATCH /clientes/{id})

Somente o próprio cliente (`hasRole('CLIENTE') and #id == authentication.principal.id`). Campos opcionais (partial update). Valida unicidade de e-mail e telefone quando informados.

Body exemplo (ClienteAtualizaDto):

```
{
  "nome": "João Silva Atualizado",
  "telefone": "(11) 91234-5678",
  "email": "joao.novo@email.com",
  "dataNascimento": "1995-05-15"
}
```

Tentar atualizar outro ID → 403 AccessDenied. E-mail/telefone já em uso → 409 DuplicateResourceException.

7. Alterar Senha (PATCH /clientes/{id}/senha)

Somente o próprio cliente. Valida senha atual, confirmação da nova senha e impede reutilizar a senha atual. Resposta de sucesso: **204 No Content**.

Body (ClienteAlteraSenhaDto):

```
{
  "senhaAtual": "Senha@123",
  "novaSenha": "NovaSenha@456",
  "confirmaNovaSenha": "NovaSenha@456"
}
```

Cenário	Status	Mensagem
Senha atual incorreta	400	Senha atual incorreta!
Nova ≠ confirmação	400	Nova senha e confirmação não conferem!
Nova = senha atual	400	A nova senha deve ser diferente da senha atual!
Nova senha fraca	400	Validação @Pattern
ID de outro cliente	403	Você só pode alterar a sua própria senha!

8. Desativar e Reativar

8.1 PATCH /clientes/desativar/{id}

Exclusão lógica (status → INATIVO). Controller restringe a CLIENTE + próprio ID. Service também permite ADMIN. Já inativo → 400 "Cliente já está desativado!". Sucesso: 204 No Content.

8.2 PATCH /clientes/{id}/reativar

Somente ADMIN (`hasRole('ADMIN')`). Já ativo → 400 "Cliente já está ativo!". Sucesso: 204 No Content.

9. Ordem Sugerida de Execução dos Testes

1. POST /clientes (cadastro válido) → capturar cliente_id
2. Login como CLIENTE → salvar token_cliente
3. Login como ADMIN → salvar token_admin
4. Login como BARBEIRO → salvar token_barbeiro
5. GET lista/ativo e lista/inativo (com tokens corretos e incorretos)
6. GET busca/nome e GET /{id}
7. PATCH atualizar dados (próprio e de outro)
8. PATCH alterar senha (cenários de sucesso e falha)
9. PATCH desativar → verificar lista/inativo
10. PATCH reativar (como admin) → verificar lista/ativo

10. Observações Técnicas e Pontos de Atenção

• **Inconsistência de Role:** reativar usa ROLE_ADMIN; demais endpoints usam ROLE_ADMINISTRADOR. Alinhe o JWT e o @PreAuthorize.

• **SecurityConfig:** alguns requestMatchers estão sem a barra inicial (ex.: clientes/lista/ativo). Isso pode fazer a regra de segurança não ser aplicada corretamente.

• **Busca por ID:** não possui @PreAuthorize específico → cai em anyRequest().authenticated().

• **Desativar:** o @PreAuthorize do controller é mais restritivo que a lógica do service (service permite ADMIN, controller não).

• **Validação de senha:** regex exige 8+ caracteres, 1 maiúscula, 1 minúscula, 1 dígito e 1 especial (@$!%*?&.#_-).

• **Telefone:** pattern aceita (XX) 9XXXX-XXXX com ou sem parênteses/hífen.

11. Scripts de Teste Úteis (Postman Tests)

Após cadastro (capturar ID):

```
pm.test('Status 201', () => pm.response.to.have.status(201));
const json = pm.response.json();
pm.collectionVariables.set('cliente_id', json.id);
```

Validação genérica de array:

```
pm.test('Status 200', () => pm.response.to.have.status(200));
pm.test('É array', () => pm.expect(pm.response.json()).to.be.an('array'));
```

Validação 204 No Content:

```
pm.test('Status 204', () => pm.response.to.have.status(204));
```

Documento gerado automaticamente a partir do código-fonte da camada de Clientes (ClienteController, ClienteService, SecurityConfiguracao e DTOs).

BarberSystem © Zheiry — Testes Postman